

Davranış Uzayı ve Davranış Geometrisi

Ömer Faruk Yavuz

May 2026

Davranış Uzayı ve Davranış Geometrisi

Yazılım mühendisliğinin üzerinde nadiren durulan, ancak en derin katmanlarından birini oluşturan mesele, bir sistemin yalnızca işleyen hâlini değil, alabileceği tüm hâlleri kavrayabilme yetisidir.

Tek Nokta ve Uzay

Bir fonksiyon tasarlanırken zihinde genellikle **tek bir** davranış canlandırılır:

`girdi -> islem -> cikti`

Örneğin ağ üzerinden bir istek gönderen bir fonksiyon ele alındığında, zihindeki tasavvur şu biçimdedir: kullanıcı bir eylem başlatır, fonksiyon çağrılır, istek sunucuya iletilir ve başarılı bir yanıt döner. Bu tasavvur **tek bir noktaya** karşılık gelir; yani tek bir senaryoyu ve tek bir yörüngeyi temsil eder.

Ne var ki söz konusu fonksiyon, gerçek dünyada bu tek nokta ile sınırlı değildir; etrafında geniş bir olasılıklar alanı uzanır. Bu alan, aşağıdaki gibi çok sayıda alternatif durumu kapsar:

- İsteğin iletimi sırasında bağlantının yarıda kesilmesi,
- Kullanıcının işlem sırasında sekmeyi kapatması,
- Ağ bağlantısının kopup yeniden kurulması,
- Aynı isteğin mükerrer biçimde gönderilmesi,
- Sunucunun zaman aşımı (timeout) bildirmesine karşın işlemin aslında başarıyla tamamlanmış olması,
- Token'ın süresi dolarken, yeniden deneme (retry) esnasında yenileme (refresh) mekanizmasının tetiklenmesi,
- Belleği aşan büyük veri yüklerinin işlenmesi,
- Mobil cihazda uygulamanın arka plana alınması.

Bu olasılıkların her biri **birer noktadır**; tamamı bir araya geldiğinde ise **bir uzay** oluşturur.

Davranış Uzayı Kavramı

Söz konusu yapıya **davranış uzayı** (*behavior space*) adı verilir; bu kavram, bir sistemin alabileceği tüm hâllerin kümesini ifade eder. Bu küme yalnızca sistemin beklenen biçimde işlediği hâli değil; kırıldığı, yarım kaldığı, çakıştığı, hatalı sıralandığı, geciktirdiği, erken döndüğü ya da hiç dönmediği tüm hâllerin toplamını içerir.

Geometri: Uzayın Şekli

“Geometri” nitelemesi tesadüfi değildir. Söz konusu uzay rastgele bir yapıya sahip olmayıp **belirli bir şekle** sahiptir. Bazı bölgeleri birbirine yakın, bazıları uzaktır; bazı bölgeler sıkça ziyaret edilirken bazılarına neredeyse hiç girilmez. Benzer biçimde sistem, kimi bölgelerde kararlı davranırken kimi bölgelerde belirsiz biçimde tutarsızlaşır.

Merkez: Mutlu Yol

“Tek kullanıcı, hızlı bağlantı, olağan akış” senaryosu uzayın merkezini oluşturur; bu merkez geniş, düz ve görece kararlı bir bölgedir. Testlerin büyük çoğunluğu da bu bölgede yürütülür.

Uzak Köşeler

“Yeniden bağlanma sırasında yeniden deneme, eş zamanlı token yenileme ve mükerrer istek” bileşimi, uzayın uzak bir köşesini temsil eder. Üç düşük olasılıklı olayın eş zamanlı gerçekleşmesiyle girilen bu dar bölgelerde sistem sıklıkla çöker; zira bu bölgeler nadiren incelenir.

Çatlaklar

“Bayatlamış durum (*stale state*) ile eş zamanlı yazma” birleşimi, uzayda bir çatlak oluşturur. Bu çatlak doğrudan görünür değildir; ancak içine düşülmesi hâlinde veri bütünlüğü bozulur.

“Geometri” kavramının tercih edilme gerekçesi de buradan ileri gelir: uzayın bir şekli, eğrilikleri, çatlakları ve uçurumları vardır. Yetkin bir mühendis kod yazarken yalnızca merkezi değil, kenarları da görmeye çalışır. Bu yetkinlik ise tecrübe gerektirir; zira çatlakların büyük çoğunluğu ancak üretim ortamında, gerçek kullanıcılarla ve beklenmedik kombinasyonlar altında kendini açığa vurur.

İnsan Geliştiricinin Doğal Eğilimi

İnsan geliştiricinin bilişsel eğilimi şu yöndedir: önce **mutlu yol** yazılır, buna göre test koşulu ve sonuçta “çalışıyor” yargısına varılır. Ancak bu yargı, esasen şu anlama gelir: “Sistem uzayın merkezinde çalışıyor; kenarlardaki davranış ise belirsizdir.”

Yapay Zekânın İşlevi

Yapay zekâ destekli araçların belirli bağlamlarda yerine getirdiği işlev — örneğin kod yazımının ardından “sinir testi yap” yönergesi verildiğinde — uzayın **kenarlarını taramaktır**. Model, “Bu kod hangi koşullarda kırılır?” sorusunu işler; çünkü eğitim verisinde sayısız olay sonrası inceleme (post-mortem) raporu, hata kaydı ve uç durum analizi bulunur. Benzer çatlakları daha önce farklı kod tabanlarında gözlemlemiş olması, ona “burayı kontrol et” önerisini sunma kapasitesi kazandırır.

Geliştirici kod yazarken **tek bir** yörüngeyi imgelerken, yapay zekâ kodu okurken **olası yörüngelerin dağılımını** değerlendirir. Bu epistemik fark son derece belirleyicidir.

Yüksek Boyutlu Uzay Metaforu

Bir sistemin davranış uzayı **yüksek boyutlu** bir yapıdır. Her girdi bir boyut, her durum değişkeni bir boyut, her zamanlama farkı ise ayrı bir boyut oluşturur. Elli boyutlu bir uzay tasavvur edildiğinde, geliştirici bu uzayda yalnızca üç boyutlu bir gözlemci konumundadır ve yalnızca yakın çevresini algılayabilir. Hataların büyük bölümü de, geliştiricinin doğrudan göremediği, ancak uzayda var olmayı sürdüren **uzak köşelerde** barınır.

İyi Mimari: Uzayın Sadeleştirilmesi

Sağlam mühendislik pratiği, esasen bu boyutların azaltılmasına dayanır. Bu azaltma, çeşitli mimari araçlarla gerçekleştirilir. Değişmez durum (*immutable state*) bir boyutu ortadan kaldırır; çünkü mutasyon olanağını eler. Saf fonksiyon (*pure function*) birden fazla boyutu ortadan kaldırır; zira yan etkiyi devre dışı bırakır. İdempotent uç nokta bir boyutu sabitler; böylece çağrı sayısı sonucu etkilemez. Durum makinesi (*state machine*) uzayı sonlu parçalara böler ve geçişleri açıkça tanımlar. Tip sistemi ise uzayı daraltır; geçersiz hâlleri daha derleme (compile) aşamasında eler.

Bu çerçevede her isabetli mimari karar, özünde “davranış uzayını sadeleştirme” hareketi olarak okunabilir: daha az köşe, daha az çatlak ve daha az uçurum.

Davranış Uzayının Köşelerinden Tipik Hâller

Bir kodun yalnızca “çalışan hâli” yoktur; aynı zamanda çok sayıda arıza hâli de mevcuttur. Bunlar arasında yarış koşulu (*race condition*), zaman aşımı (*time-out*), kısmi başarısızlık (*partial failure*), çift tıklama (*double click*), bayatlamış önbellek (*stale cache*), WebSocket yeniden bağlanması, token yenileme çakışması (*overlap*), çevrimdışı–çevrimiçi geçişi, animasyon kesintisi (*interrupt*) ve bozulmuş durum (*corrupted state*) hâlleri sayılabilir. Bu hâllerin büyük çoğunluğu ise yaşanmadan tam olarak kavranamaz.

Test Türleri: Uzak Tarama Yöntemleri

Davranış uzayının farklı bölgeleri, farklı test yöntemleriyle taranır. Sinir testi (*nerve test*), sistemi kasten rahatsız ederek tepkisini gözlemlemeyi amaçlar. Stres testi, merkezden uzağa doğru artan bir yük bindirir. Maymun testi (*monkey test*), rastgele noktalara müdahale eder. Sabırsız kullanıcı testi, gerçekçi bir kullanıcı sabırsızlığını taklit eder. Kaos testi (*chaos engineering*) ise altyapıyı kasten bozarak uzayın dayanıklılığını ölçer.

Yapay Zekânın Yeni Konumu

Bu çerçevede dikkat çeken nokta şudur: yapay zekâ artık yalnızca bir “kod tamamlayıcı” araç olmaktan çıkmaktadır. Bunun yerine giderek davranış denetçisi, sistem stresi gözlemcisi, uç durum avcısı ve mimari rezonans okuyucusu gibi rollere bürünmektedir.